

<주제>

F1 pit stop 예측하기

<코드 흐름>

1. 전처리, 피쳐 엔지니어링

- 불필요한 칼럼 삭제함
- feature_engineering: 마모 관련 추가 변수 생성

```
def feature_engineering(df, fit=False):  
    df['_LapNumber_/_RaceProgress'] = (df['LapNumber'] /  
                                         (df['RaceProgress'] + 1e-6)).astype('float32')  
    df['_TyreLife_/_LapNumber'] = (df['TyreLife'] /  
                                    df['LapNumber'].clip(lower=1)).astype('float32')
```

- 인코딩: np.floor & factorize

```
for col in num_cols + ['_LapNumber_/_RaceProgress', '_TyreLife_/_LapNumber']:  
    cat_name = f"{col}_cat_" if col in num_cols else f"{col[1:]}_cat_"  
    if fit:  
        codes, uniques = np.floor(df[col]).factorize()  
        category_map[col] = uniques  
    else:  
        uniques = category_map[col]  
        code_map = {cat: i for i, cat in enumerate(uniques)}  
        codes = np.floor(df[col]).map(code_map).fillna(-1).astype('int32')  
    df[cat_name] = codes  
    df[cat_name] = df[cat_name].astype(str)
```

```
for col in cat_cols + ['Year_cat_', 'PitStop_cat_']:  
    count_name = f"_{col}_count" if col in cat_cols else f"_{col[:-1]}_count"  
    if fit:  
        count_map = df[col].value_counts()  
        category_map[count_name] = count_map  
    else:  
        count_map = category_map[count_name]  
    df[count_name] = df[col].map(count_map).fillna(0).astype('int32')
```

2. 하이퍼 파라미터 설정

- CFG클래스: 설정 종합 관리용
- Params: RealMLP 모델의 설정 정의

3. kFold교차 검증 학습

- StratifiedKFold: 데이터 5개로 나눔, 안정화 시킴
- Target encoding
- Train

4. overall out of fold 예측

- 전체 데이터에 대한 성능 계산

<새롭게 알게 된 내용/어려운 내용/배울 점>

알게 된 내용: 기존 데이터들을 기반으로 타이어 마모에 대한 추가 변수를 생성한 것이 인상깊었다.

어려운 내용: target encoding이 어려웠다. 범주형 변수를 숫자로 바꿀 때 해당 범주형 변수의 y값의 평균으로 바꿨기 때문

Feature engineering 할 때 category_map을 따로 만들어서 train 전처리를 하고, 또 같은 기준으로 test 전처리를 했는데 그 과정이 이해하기에 복잡했다.

배울 점: 지금까지 데이터 분석할 때 추가 변수는 만들어본 적이 없는데, 해당 수상작에서 비율을 추가 변수로 넣은 것을 보고 보고 배워야겠다고 생각했다. 그리고 범주형 변수를 수치형으로 바꿀 때 target mean으로 바꿨던데, 이 부분도 좋은 예측 성능이 나오는데 일조했다고 생각했다.